

Data Storage Layer - MongoDB NoSQL

Big Data Analytics Final Project

Team: Emre Akyol, Harmanpreet Chauhan, Mohamed Nasr

This notebook implements the NoSQL storage layer using MongoDB, demonstrating document-based storage, indexing strategies, and aggregation pipelines.

```
In [1]: import pymongo
from pymongo import MongoClient, ASCENDING, DESCENDING
import json
import os
from datetime import datetime
import pandas as pd

print(f"PyMongo version: {pymongo.__version__}")
print(f>Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')})
```

PyMongo version: 4.6.1
Date: 2026-01-04 22:04:26

Connect to MongoDB

```
In [2]: # Connect to MongoDB (adjust URI as needed)
MONGO_URI = os.environ.get('MONGODB_URI', 'mongodb://localhost:27017/')

try:
    client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
    client.server_info() # Test connection
    print(f"Connected to MongoDB at {MONGO_URI}")

    db = client['crypto_sentiment']
    print(f"Database: crypto_sentiment")
except Exception as e:
    print(f"MongoDB connection failed: {e}")
    print("\nTo start MongoDB with Docker:")
    print("  docker run -d -p 27017:27017 --name crypto-mongodb mongo:7.0")
```

Connected to MongoDB at mongodb://localhost:27017/
Database: crypto_sentiment

Create Collections and Indexes

```
In [3]: # Drop existing collections for fresh start
for coll in ['crypto_prices', 'sentiment_data', 'market_overview']:
    if coll in db.list_collection_names():
        db[coll].drop()
        print(f"Dropped existing collection: {coll}")
```

```

# Create collections
crypto_prices = db['crypto_prices']
sentiment_data = db['sentiment_data']
market_overview = db['market_overview']

print("\nCollections created")

```

Collections created

```

In [4]: # Create indexes for optimized queries
print("Creating indexes...\n")

# Crypto prices indexes
crypto_prices.create_index('symbol', unique=True)
crypto_prices.create_index([('timestamp', DESCENDING)])
crypto_prices.create_index([('symbol', ASCENDING), ('timestamp', DESCENDING)])
crypto_prices.create_index([('volatility', DESCENDING)])
crypto_prices.create_index([('socialSentiment', DESCENDING)])
print("crypto_prices indexes: symbol, timestamp, volatility, socialSentiment")

# Sentiment data indexes
sentiment_data.create_index([('timestamp', DESCENDING)])
print("sentiment_data indexes: timestamp")

# Market overview indexes
market_overview.create_index([('timestamp', DESCENDING)])
print("market_overview indexes: timestamp")

print("\nIndexes created successfully")

```

Creating indexes...

```

crypto_prices indexes: symbol, timestamp, volatility, socialSentiment
sentiment_data indexes: timestamp
market_overview indexes: timestamp

```

Indexes created successfully

Load and Store Data

```

In [5]: # Load cryptocurrency data
with open('./resources/data/crypto-prices.json', 'r') as f:
    crypto_data = json.load(f)

print(f"Loaded {len(crypto_data.get('cryptocurrencies', []))} cryptocurrencies")
print(f"Market cap: ${crypto_data['marketOverview']['totalMarketCap']/1e12:.2f}T")

```

```

Loaded 6 cryptocurrencies
Market cap: $3.21T

```

```

In [6]: # Store cryptocurrency data
print("Storing cryptocurrency data...\n")

for crypto in crypto_data['cryptocurrencies']:
    # Ensure all numeric values are proper Python floats

```

```

doc = {
    'symbol': crypto['symbol'],
    'name': crypto['name'],
    'price': float(crypto['price']),
    'marketCap': float(crypto['marketCap']),
    'volume24h': float(crypto['volume24h']),
    'change24h': float(crypto['change24h']),
    'volatility': float(crypto['volatility']),
    'socialSentiment': float(crypto.get('socialSentiment', 0)),
    'buzzVolume': int(crypto.get('buzzVolume', 0)),
    'timestamp': datetime.now()
}

result = crypto_prices.update_one(
    {'symbol': crypto['symbol']},
    {'$set': doc},
    upsert=True
)

status = "Inserted" if result.upserted_id else "Updated"
print(f" {status} {crypto['symbol']}: ${doc['price']:,.2f}")

print(f"\nStored {crypto_prices.count_documents({})} documents")

```

Storing cryptocurrency data...

```

Inserted BTC: $91,241.00
Inserted ETH: $3,138.92
Inserted XRP: $2.09
Inserted SOL: $133.91
Inserted DOGE: $0.15
Inserted ADA: $0.40

```

Stored 6 documents

```

In [7]: # Store market overview
market_doc = {
    'totalMarketCap': float(crypto_data['marketOverview']['totalMarketCap']),
    'totalVolume': float(crypto_data['marketOverview']['totalVolume']),
    'btcDominance': float(crypto_data['marketOverview']['btcDominance']),
    'fearGreedIndex': int(crypto_data['marketOverview']['fearGreedIndex']),
    'socialSentiment': float(crypto_data['marketOverview']['socialSentiment']),
    'timestamp': datetime.now()
}

market_overview.insert_one(market_doc)
print("Stored market overview")

```

Stored market overview

```

In [8]: # Store sentiment data
try:
    with open('./resources/data/sentiment-data.json', 'r') as f:
        sent_data = json.load(f)

    sentiment_doc = {
        'fearGreedIndex': sent_data['sentimentOverview']['fearGreedIndex'],

```

```

        'classification': sent_data['sentimentOverview']['classification'],
        'newsCount': len(sent_data.get('newsHeadlines', [])),
        'headlines': sent_data.get('newsHeadlines', [])[:10], # Store top 10
        'timestamp': datetime.now()
    }

    sentiment_data.insert_one(sentiment_doc)
    print(f"Stored sentiment data ({sentiment_doc['newsCount']} headlines)")
except FileNotFoundError:
    print("Sentiment data file not found, skipping")

```

Stored sentiment data (0 headlines)

Query Examples

```

In [9]: # Query 1: Get all cryptocurrencies sorted by market cap
print("Cryptocurrencies by Market Cap:\n")
results = crypto_prices.find().sort('marketCap', DESCENDING)
for doc in results:
    print(f" {doc['symbol']}: ${doc['marketCap']/1e9:.1f}B")

```

Cryptocurrencies by Market Cap:

```

BTC: $1822.5B
ETH: $378.9B
XRP: $126.8B
SOL: $75.5B
DOGE: $25.2B
ADA: $14.7B

```

```

In [10]: # Query 2: Find high volatility cryptocurrencies
print("High Volatility (>2%):\n")
high_vol = crypto_prices.find({'volatility': {'$gt': 0.02}})
for doc in high_vol:
    print(f" {doc['symbol']}: {doc['volatility']*100:.2f}%")

```

High Volatility (>2%):

```

DOGE: 5.36%
XRP: 4.19%
ADA: 2.80%

```

```

In [11]: # Query 3: Find positive sentiment cryptocurrencies
print("Positive Sentiment Cryptocurrencies:\n")
positive = crypto_prices.find({'socialSentiment': {'$gt': 0}}).sort('socialSentimen
for doc in positive:
    print(f" {doc['symbol']}: {doc['socialSentiment']:.3f}")

```

Positive Sentiment Cryptocurrencies:

```

DOGE: 0.120
XRP: 0.080
ADA: 0.060
SOL: 0.040
ETH: 0.030
BTC: 0.020

```

Aggregation Pipelines

```
In [12]: # Aggregation: Calculate average metrics
pipeline = [
    {'$group': {
        '_id': None,
        'avgPrice': {'$avg': '$price'},
        'avgVolatility': {'$avg': '$volatility'},
        'avgSentiment': {'$avg': '$socialSentiment'},
        'totalVolume': {'$sum': '$volume24h'},
        'count': {'$sum': 1}
    }}
]

result = list(crypto_prices.aggregate(pipeline))
if result:
    r = result[0]
    print("Aggregated Metrics:")
    print(f"  Average Price: ${r['avgPrice']:, .2f}")
    print(f"  Average Volatility: {r['avgVolatility']*100: .2f}%")
    print(f"  Average Sentiment: {r['avgSentiment']: .3f}")
    print(f"  Total Volume: ${r['totalVolume']/1e9: .2f}B")
    print(f"  Count: {r['count']}")
```

Aggregated Metrics:
Average Price: \$15,752.74
Average Volatility: 2.63%
Average Sentiment: 0.058
Total Volume: \$54.86B
Count: 6

```
In [13]: # Aggregation: Volatility classification
pipeline = [
    {'$project': {
        'symbol': 1,
        'volatility': 1,
        'category': {
            '$switch': {
                'branches': [
                    {'case': {'$lt': ['$volatility', 0.02]}, 'then': 'Low'},
                    {'case': {'$lt': ['$volatility', 0.05]}, 'then': 'Medium'}
                ],
                'default': 'High'
            }
        }
    }},
    {'$group': {'_id': '$category', 'cryptos': {'$push': '$symbol'}, 'count': {'$sum': 1}}
]

print("Volatility Classification:\n")
for doc in crypto_prices.aggregate(pipeline):
    print(f"  {doc['_id']}: {'', '.join(doc['cryptos'])}")
```

Volatility Classification:

High: DOGE
Medium: XRP, ADA
Low: BTC, ETH, SOL

Database Statistics

```
In [14]: print("Database Statistics:\n")
stats = db.command('dbStats')
print(f"  Database: {stats['db']}")
print(f"  Collections: {stats['collections']}")
print(f"  Documents: {stats['objects']}")
print(f"  Storage Size: {stats['storageSize']/1024:.2f} KB")
print(f"  Indexes: {stats['indexes']}")

print("\nCollection Counts:")
for coll in ['crypto_prices', 'sentiment_data', 'market_overview']:
    count = db[coll].count_documents({})
    print(f"  {coll}: {count} documents")
```

Database Statistics:

Database: crypto_sentiment
Collections: 3
Documents: 8
Storage Size: 12.00 KB
Indexes: 10

Collection Counts:

crypto_prices: 6 documents
sentiment_data: 1 documents
market_overview: 1 documents

```
In [15]: # Export for verification
export_data = list(crypto_prices.find({}, {'_id': 0}))
df = pd.DataFrame(export_data)
print("Stored Data Preview:\n")
display(df[['symbol', 'price', 'change24h', 'volatility', 'socialSentiment']])
```

Stored Data Preview:

	symbol	price	change24h	volatility	socialSentiment
0	BTC	91241.000000	1.04602	0.010460	0.02
1	ETH	3138.920000	0.82503	0.008250	0.03
2	XRP	2.090000	4.19362	0.041936	0.08
3	SOL	133.910000	1.54099	0.015410	0.04
4	DOGE	0.149850	5.35820	0.053582	0.12
5	ADA	0.398885	2.79540	0.027954	0.06

Summary

MongoDB storage layer implemented with:

- 3 collections: crypto_prices, sentiment_data, market_overview
- Optimized indexes for common queries
- Aggregation pipelines for analytics
- Upsert operations for data updates